Guide d’installation — Demo 1 MaaS (RHOAI 3.5)

Ce guide permet de **reproduire la Demo 1 sur n’importe quel sandbox OpenShift** disposant de Red Hat OpenShift AI 3.5.

Objectif de la Demo 1 : publier un modèle via le chemin **natif** :

```
LLMInferenceService (llm-d) → MaaSModelRef → modèle visible dans MaaS
```

Hors scope (Demo 2) : abonnements, clés API, quotas, révocation.

---

1. Ce dont tu as besoin

Élément	Détail
Sandbox OpenShift	Temporaire / jetable (ex. OpenTLC)
RHOAI	Version **3.5.x** déjà installée (ou installable)
Droits	Compte **cluster-admin** (ou équivalent)
Outils locaux	`oc` , `git` , `bash` , `envsubst` (souvent via `gettext`)
Réseau	Accès pull vers `registry.redhat.io` et `ghcr.io`

Tu n’as **pas** besoin de coller ton token dans Git, ni dans un fichier du dépôt.

---

2. Récupérer le dépôt

```
git clone https://github.com/Lindagh1/rhoai-maas-demo.git
cd rhoai-maas-demo
git checkout main
```

Si tu es déjà dans le dépôt :

```
git pull
```

---

3. Se connecter au sandbox

1. Ouvre la console OpenShift du sandbox.
2. Clique sur ton nom (en haut à droite) → **Copy login command**.
3. Dans **ton terminal** (pas dans le chat) :

```
oc login --server="https://api.<TON-SANDBOX>:6443" --token="<TON-TOKEN>" --insecure-skip
```

Vérifie que tu es au bon endroit :

```
oc whoami
oc whoami --show-server
```

Le serveur doit contenir l'identifiant de **ton** sandbox. Si ce n'est pas le cas : **arrête**.

---

4. Auditer (lecture seule)

```
bash scripts/00-audit.sh
```

Lis ensuite :

- `docs/sandbox-audit-raw.txt` (brut, local, non versionné)
- ou refais les checks du `docs/sandbox-audit.md` comme modèle

Points critiques à noter :

1. Version OpenShift / RHOAI
2. GPU libre ou non (souvent **1 GPU déjà pris** par un modèle existant)
3. Présence ou non de RHCL / MaaS / Gateway `maas-default-gateway`

---

5. Installer la Demo 1 (idempotent)

Exécute **dans l'ordre** :

```
bash scripts/10-prerequisites.sh   # RHCL, Kuadrant, UWM, GatewayClass, TLS Authorino
bash scripts/20-enable-maas.sh     # Gateway MaaS, Postgres, activation aigateway
bash scripts/30-deploy-native-model.sh   # LLMInferenceService (simulateur CPU)
bash scripts/40-publish-maas.sh    # MaaSModelRef (publication déclarative)
```

Puis vérifie **deux fois** (idempotence) :

```
bash scripts/50-verify.sh
bash scripts/50-verify.sh
```

Que font ces scripts ?

Script	Rôle
`10`	Prérequis plateforme manquants seulement
`20`	Active MaaS (sans casser les Gateways déjà là)
`30`	Déploie le modèle natif dans `rhoai-maas-demo`
`40`	Publie via `MaaSModelRef` (équivalent du bouton « Publish as MaaS »)
`50`	Checklist de santé
`99`	Nettoyage **uniquement** des ressources de la démo (`CONFIRM=yes`)

Les scripts sont **ré-exécutables** : tu peux les relancer sans tout casser.

---

6. Ce qui est normal / pas un échec

GPU déjà occupé

Beaucoup de sandboxes n'ont **qu'un seul GPU L4**, déjà utilisé par un `InferenceService` legacy (ex. `llama-32-3b-instruct`).

Dans ce cas, la Demo 1 déploie un **simulateur CPU** (`llm-d-inference-sim`) :

- même chemin natif `LLMInferenceService` → `MaaSModelRef`
- pas de concurrence GPU
- ne touche **pas** au modèle legacy existant

`MaaSModelRef` en Pending

Sur RHOAI 3.5.2, le statut global `Ready` du `MaaSModelRef` attend souvent une **paire** :

- `MaaSSubscription`
- `MaaSAuthPolicy`

→ c'est la **Demo 2**.
Pour la Demo 1, on valide surtout :

- `LLMInferenceService` **Ready**
- `MaaSModelRef` présent avec `kind: LLMInferenceService`
- condition **`RuntimeReady=True`** (« Backend is healthy »)

---

7. Vérifications rapides (CLI)

```
oc get dsc default-dsc -o jsonpath='{.status.conditions[?(@.type=="ModelsAsAServiceReady
oc get llminferenceservice -n rhoai-maas-demo
oc get maasmodelref -n rhoai-maas-demo
oc get gateway maas-default-gateway -n openshift-ingress
```

Dashboard :

```
https://rhods-dashboard-redhat-ods-applications.<domaine-apps-du-cluster>
```

---

8. Nettoyage (optionnel)

```
CONFIRM=yes bash scripts/99-cleanup.sh
```

Cela **ne désinstalle pas** RHOAI ni RHCL (plateforme partagée).
Cela **ne touche pas** au projet `my-first-model` s'il existe.

---

9. En cas de problème

Voir `troubleshooting.md` .

Bloqueurs typiques :

- mauvais cluster / pas admin
- catalogues Operators indisponibles
- pull `ghcr.io` bloqué
- MaaS non Ready (Postgres / secret `maas-db-config` / UWM)

---

10. Enchaînement recommandé avant une démo client

1. Nouveau sandbox → `oc login`
2. `00-audit` → lire les alertes GPU
3. `10` → `40` → `50` × 2
4. Ouvrir le dashboard et **répéter une fois** le parcours du guide démo live
5. Seulement ensuite : présentation client